数据结构 A 第 2 次作业

姓名: 王子川　　**学号:** PB25000055

1(2.12) 顺序表的比较

设 $k = \min(A.length, B.length)$, 对于表 A,B 前长为 k 的部分, 用一个变量 i 进行遍历, 如果程序没有终止, 说明 A,B 前面长为 k 的序列完全相等, 因此只需要比较 A,B 的表长即可.

```
int CompareList(SqList A, SqList B) {
    int la = A.length; int lb = B.length;
    int k = min(la, lb); // length of the common comparable part

    for (int i = 1; i <= k; i++) {
        if (A.elem[i] < B.elem[i]) return -1;
        if (A.elem[i] > B.elem[i]) return 1;
    }

    if (la < lb) return -1; // A is a proper prefix of B
    if (lb < la) return 1;  // B is a proper prefix of A
    return 0;
}
```

2(2.21) 顺序表的逆置

变量 i 从 1 开始向右遍历, 变量 j 从 A.length 开始向左遍历, 依次交换 A 中元素, 直到 $i \geq j$ 时停止.

```
Status InverseList(SqList& A) {
    int i = 1; // left pointer
    int j = A.length; // right pointer
    ElemType tmp;

    while (i < j) {
        // swap A.elem[i] and A.elem[j]
        tmp = A.elem[i];
        A.elem[i] = A.elem[j];
        A.elem[j] = tmp;
        // move the pointer
        i++; j--;
    }
    return OK;
}
```

3(2.25) 有序表的交集

变量 i,j,k 分别表示指向有序表 A,B,C 当前位置的指针, 每轮循环中保证 i,j 指针至少有一个向前移动. 如果当前位置 $A.elem[i] = B.elem[j]$, 就把当前元素插入表 C, i 和 j 指针一起向前移动一个元素, 如果 $A.elem[i] < B.elem[j]$, 将 i 指针向前移动, 否则将 j 指针向前移动.

```
Status IntersectList(SqList A, SqList B, SqList& C) {
    int i = 1; // current position in A
    int j = 1; // current position in B
    int k = 0; // number of elements in C
    int la = A.length;
    int lb = B.length;

    while (i <= la && j <= lb) {
        ElemType a = A.elem[i];
        ElemType b = B.elem[j];
    // common element
        if (a == b) {
            ListInsert(C, ++k, a);
            i++;
            j++;
        }
        // A.elem[i] cannot appear later in B
        else if (a < b)  i++;
        // B.elem[j] cannot appear later in A
        else j++;
    }
    return OK;
}
```

4(2.29) 有序表删除相同元素

i,j,k 分别表示指向表 A,B,C 当前位置的指针, p 表示 A 中当前已保留元素的个数, 同时指向保留部分的末尾位置. 每次循环保证 i,j,k 指针至少一个向前移动一个元素, 循环到 i 或 j 或 k 超过表长终止. 如果当前 $B.elem[j] = C.elem[k]$, 考察 $A.elem[i]$, 若三者相等, 则 A 表需去掉当前元素, 只移动 i 指针而不移动 p 指针; 若 $A.elem[i]$ 小于二者, 则保证当前元素 $A.elem[i]$ 一定不在 $B \cap C$ 中, 于是将 $A.elem[i++]$ 赋值给 $A.elem[++p]$; 否则若 $A.elem[i]$ 大于二者, 则需同时向前移动 j,k 指针. 如果 $B.elem[j] < C.elem[k]$, 向前移动 j 指针. 否则向前移动 k 指针. 循环结束后将 A 中剩余的元素全部赋值到前面.

算法的时间复杂度一定是线性在 A,B,C 的表长的, 因为每轮循环的复杂度是 $O(1)$, 两个循环都能保证每轮循环 i,j,k 指针至少一个向前移动一个元素, 因此算法的时间复杂度是

$$O(A.length + B.length + C.length).$$

```
Status DeleteCommonElements(SqList& A, SqList B, SqList C){
    int la = A.length, lb = B.length, lc = C.length;
    int i = 1; // pointer to A
    int j = 1; // pointer to B
    int k = 1; // pointer to C
    int p = 0; // position for retained elements in A

    while (i <= la && j <= lb && k <= lc) {
        if (B.elem[j] == C.elem[k]) {
            if (A.elem[i] == B.elem[j]) i++;
            else if (A.elem[i] < B.elem[j]) A.elem[++p] = A.elem[i++];
            else { j++; k++; }
        }
        else if (B.elem[j] < C.elem[k]) j++;
        else k++;
    }
    while (i <= la) A.elem[++p] = A.elem[i++];
    A.length = p;

    return OK;
}
```